

DFCD Cyberdeck — Assembly & Commissioning Guide

Companion to the DesignatedFreeCadDevice (ArcticEnrichmentCenter) • parts in `hardware/`, OS in `os/`, purchasing in `SHOPPING.md` • figures are schematic — dimensional truth lives in the CAD, and where a figure simplifies, the text states the governing detail explicitly.

This guide assumes working fluency with hand tools, soldering, and basic DC electronics. Each step lists prerequisites, the operation, and an acceptance check — do not advance past a failed check, since downstream steps assume the prior subassembly is mechanically and electrically sound.

Step 1 — Fabricate the printed parts

Printer ≥ 200×200 mm bed • PETG (preferred) or PLA • 0.4 mm nozzle

Print everything under `hardware/DFCD mesh files/`, preserving the folder hierarchy as your parts inventory. Use 4 perimeters and 40% infill on load-bearing members (chassis halves, bearing housings, picatinny rails and clamps) — these carry the sling loads and the cantilever moment of the raised screen. Cosmetic parts (vents, lids, buttons) are fine at 2 perimeters / 15%. The dragchain needs *dimensional discipline*: print segments flat, no supports in the pin bores, and dry-fit two links before committing to the full run of ~10–14 segments + 2 end pieces. If you are fitting the GPIO-side RTC or taller header hardware, substitute `hardware-custom/Screen frame/GFPIO lid raised 12mm.3mf` for the stock lid; and to mount the front status LED, print `hardware-custom/Screen frame/Righth screen frame with LED.3mf` in place of the stock right screen frame (it adds a 3 mm LED hole between the front buttons — see Step 6).

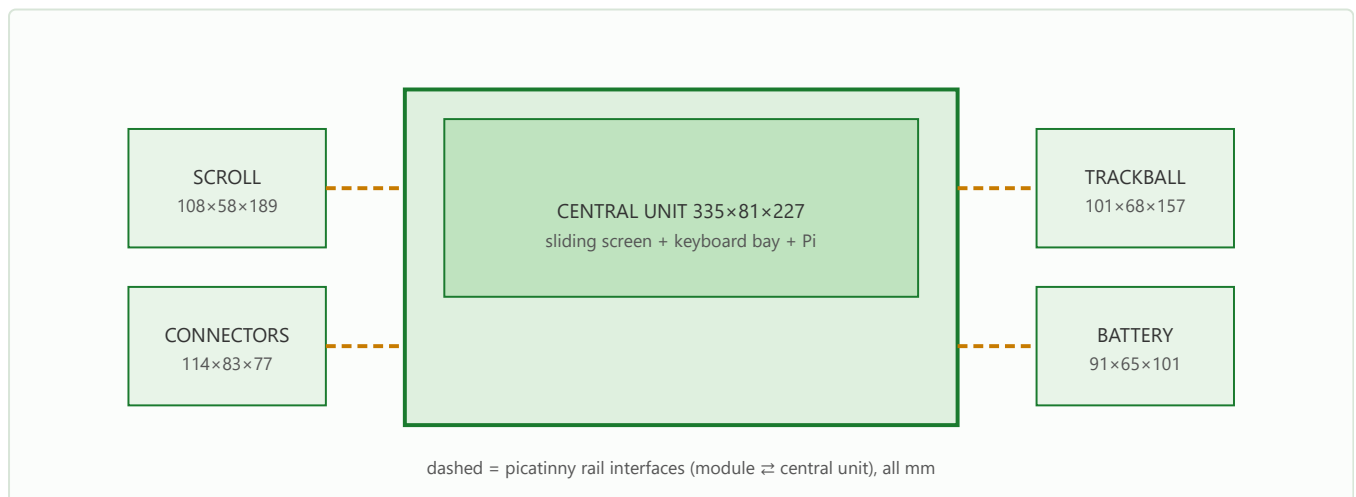


Fig. 1 — Topology of the five assemblies. Modules clamp to bottom rails; USB runs along the rear.

Check: chassis halves mate without gap along the parting line; a dragchain pair articulates through ~120° without binding.

Step 2 — Hardware preparation: inserts, bearings, linear rails

Soldering iron (heat-set tip) • M2.5/M3 brass inserts • 2× 8 mm rod • LM8UU-type bearings

Install heat-set inserts into every printed boss that will see repeated fastening — chassis screw points, bearing-housing caps, module shells. Drive each insert square with light pressure at $\sim 230^\circ\text{C}$ for PETG and let the polymer reflow around the knurling; a tilted insert produces a crooked fastener axis you cannot recover later. Press the linear bearings into the printed bearing housings (left and right) — this is an *interference fit*; if yours prints loose, a drop of CA on the outer race is acceptable since the race is static. Cut the two 8 mm rods to equal length, deburr the ends, and verify each rod slides through its bearing pair with no perceptible radial play. The screen's kinematics depend on these two rods staying parallel — the printed rod clamps set that geometry, so torque them evenly.

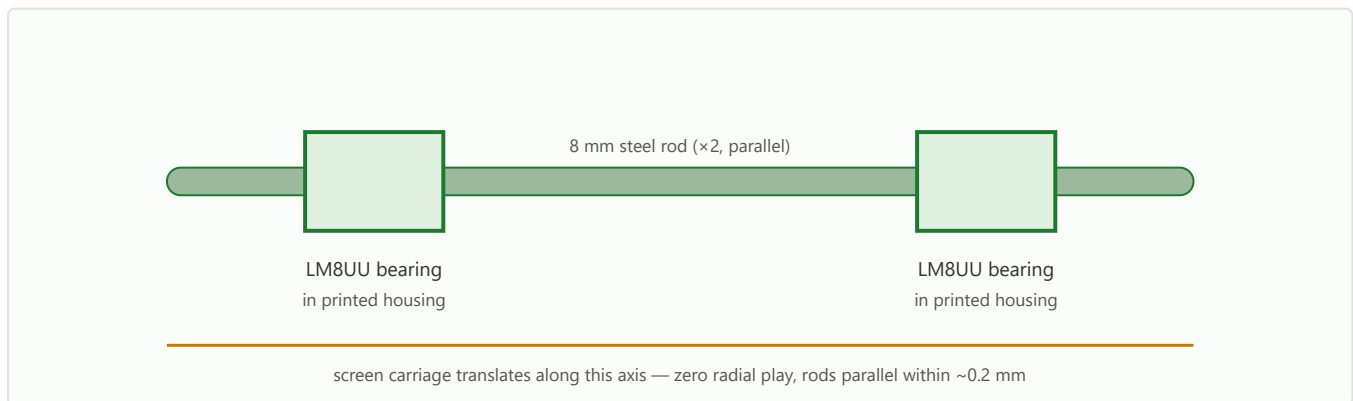


Fig. 2 — Linear slide: rod + recirculating-ball bearing in each printed housing.

Check: carriage glides end-to-end under its own weight when tilted $\sim 30^\circ$, with no chatter.

Step 3 — Central chassis assembly

M3 hardware • threadlocker (optional) • the two lower + four upper chassis prints

Join the lower chassis halves left/right, then stack the four upper chassis sections; fastener bosses register each section against its neighbour. Install the rear vents and the two bearing housings with their caps — the housings clamp the slide rods vertically, so this is where Step 2's parallelism is locked in. Mount the picatinny rails (left/right USB rails + bottom rails). The rails are structural *and* the wiring conduit: confirm the cable channel on the rear face is clear before final torque. Fit the screen-release lever and its spring; actuation should be positive with a crisp detent return.

Check: chassis sits flat, no rocking; release lever cycles 20× without fatigue whitening at the pivot.

Step 4 — Screen carriage, panel, and compute installation

10.1" panel • Pi (4B/5) + active cooler • Joy-IT power board • micro-HDMI ribbon

Assemble the screen frame halves around the 10.1" panel with its driver PCB, add the screen vents and base; mount the antenna and its release. The Pi rides behind the panel under the GPIO lid — install the active cooler first, then fasten the Pi, then the power-management board. Route the micro-HDMI 90° ribbon from the Pi to the panel driver and the USB lead for touch; dress both with strain relief at the first anchor point, since these flex with every slide cycle. The HDMI ribbon's minimum bend radius (~8 mm) is the governing constraint on routing — a sharper fold will fracture conductors intermittently, which presents later as maddening "sometimes no display" faults. If you fitted the raised GPIO lid, the extra 12 mm above the header is where a DS3231 RTC or an IDC breakout lives.

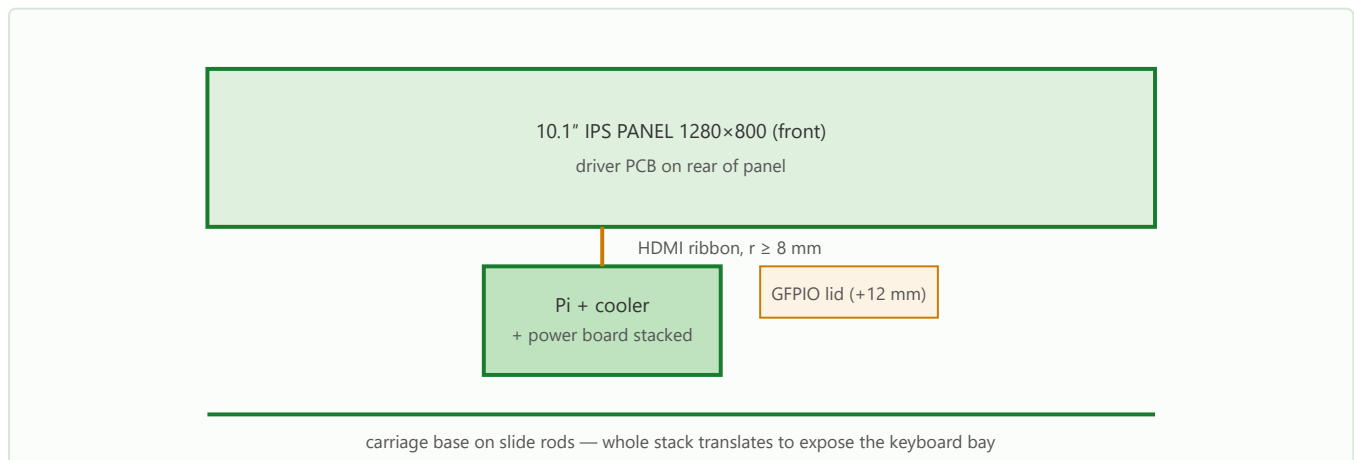


Fig. 3 — Screen stack, viewed from the side. The Pi lives behind the panel, GPIO accessible via the lid.

Check: bench-power the Pi (5 V USB-C, before battery wiring) — panel lights, touch responds.

Step 5 — Power chain: battery module → regulation → Pi

NP-F970 • pogo pins • 2B connector • Joy-IT board • multimeter

Assemble the battery module: receiver plate, contact holder with the two pogo pins, release latch, casing. The pogo pins are the only battery interface — set their preload so the NP-F lands with firm, wiping contact. Wire pogo (+/–) through the 2B self-locking connector into the chassis, then to the Joy-IT board's input. The board takes the pack's nominal 7.2 V (8.4 V fresh, ~6.0 V depleted) and regulates to 5.1 V for the Pi. **Verify polarity at the 2B connector with a meter before first connection — the pogo pad layout is mirror-symmetric, and a reversed pack survives nothing downstream.** Budget: Pi 5 under FreeCAD or sustained inference draws ~10–12 W; a 74 Wh NP-F970 yields ~5–6 h of real work.

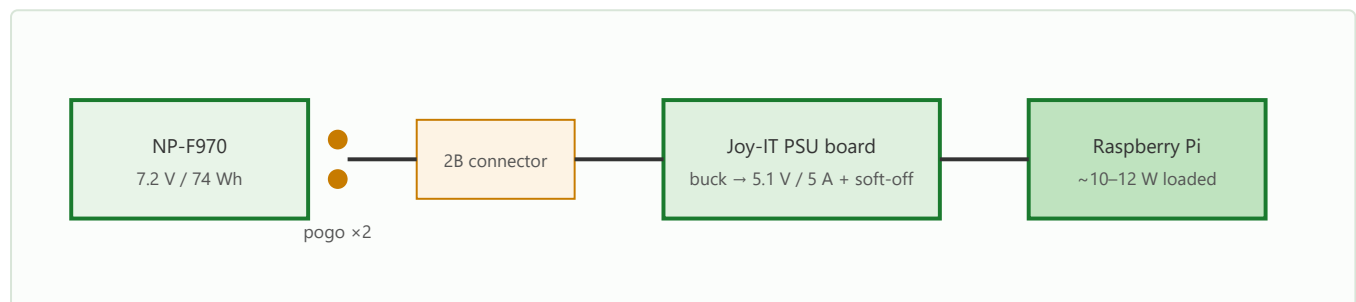


Fig. 4 — Power chain. Regulation happens in the chassis, not in the battery module.

Check: 5.06–5.2 V at the Pi input under load; latch retains the pack against a firm tug.

Step 6 — GPIO wiring: encoder, buttons, status LED, RTC

EC11 encoder • 16 mm switches ×3 • 3 mm LED + 330 Ω • DS3231 (Pi 4) or J5 battery (Pi 5)

All panel controls terminate on the 40-pin header. Suggested map (BCM numbering, matching the shipped `config.txt` overlays — change either side consistently): encoder A→GPIO17, B→GPIO27, push→GPIO22; the three screen-side buttons → GPIO23/24/25, all switching to ground with internal pull-ups (no external resistors needed). Status LED: it press-fits into the 3 mm hole in the custom `Righth` screen frame with LED part — sited in the solid web *between the bottom and middle front buttons* (X275.5, Z-35.6 in the model), so the indicator sits right in the control column. Anode to GPIO26 through 330 Ω, cathode to ground — the kernel's `gpio-led` binding drives it, and `deck-mode` selects heartbeat / off / disk-activity. RTC: on a Pi 5 simply plug the battery into J5; on a Pi 4 seat a DS3231 on pins 1–9 (3V3, SDA, SCL, GND) and uncomment `dtoverlay=i2c-rtc,ds3231`. Keep encoder leads short and twisted — quadrature edges are fast and the pair is susceptible to crosstalk from the adjacent HDMI ribbon.

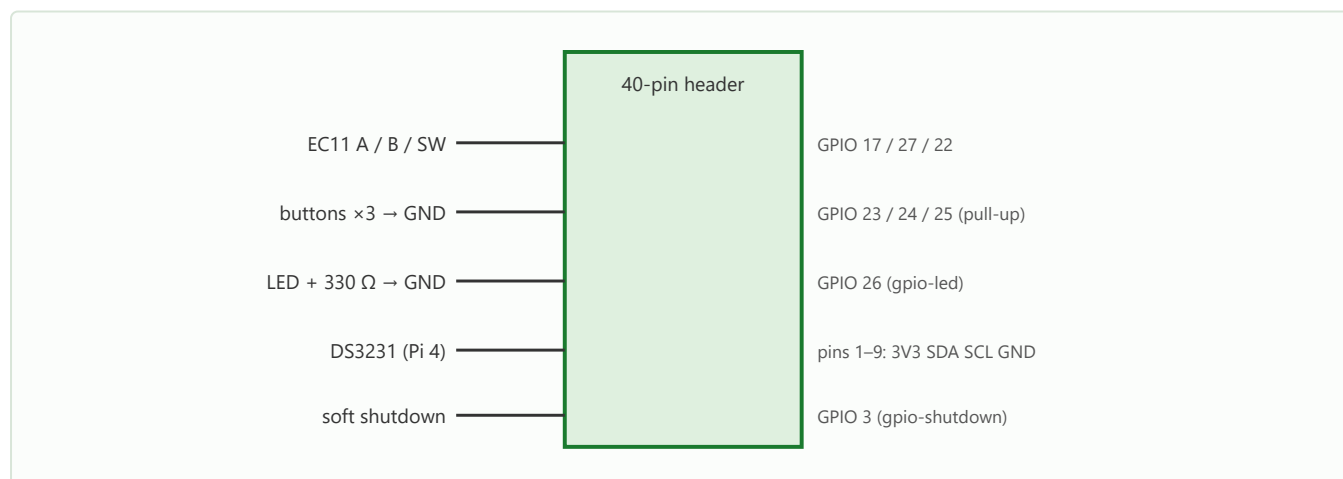


Fig. 5 — GPIO allocation. Every line ships pre-configured (commented) in the CYBERDECK-CONFIG block.

Check: after enabling overlays — `evtest` shows encoder REL events and button keys; `ls /sys/class/leds/deckstatus` exists; `timedatectl` reports RTC time.

Step 7 — USB backbone and dragchain

USB leads • Y2M 8-pin connectors • 0B connectors ×6 • assembled dragchain

The deck's modules are all USB devices. Cabling runs along the rear: each rail module breaks out through a self-locking 0B connector to the rail bus, and the two Y2M 8-pin connectors carry the aggregate across the chassis. The dragchain spans the moving joint between chassis and screen carriage, carrying the HDMI ribbon and one USB pair — lay the cables loose in the chain (no zip ties inside the links) so the conductors can find their neutral axis as the chain rolls. The trackball PCB (harvested from the Marble) and keyboard plug into the rail bus; the external-connector module simply extends two Pi ports to panel-mount sockets.

Check: with the screen cycled 50× through full travel, `lsusb` remains stable — enumeration flapping means a conductor is being worked at its bend limit.

Step 8 — Modules, rails, and the sling

Module shells • picatinny clamps • 2-point sling with QD hardware

Close up the four modules and clamp them to the rails: battery + trackball right, scroll + connectors left. Each printed clamp has a lever, pivot pin, and grip — the clamp should seat with a firm lever throw, then resist sliding under a 2 kg pull. Thread the sling through the *Strap mount* (scroll module) and *Sling mount* (trackball module) anchors so the load path runs through both forward modules into the rails — the strap anchors are upstream parts, already in your print inventory. Use a 38–50 mm webbing 2-point sling with QD hooks so the deck can be dropped to a bench without unthreading anything. The anchors are printed polymer: derate accordingly. Print them solid (100% infill, 4 perimeters) and inspect the layer adhesion before trusting them with a ~2 kg instrument at shoulder height.

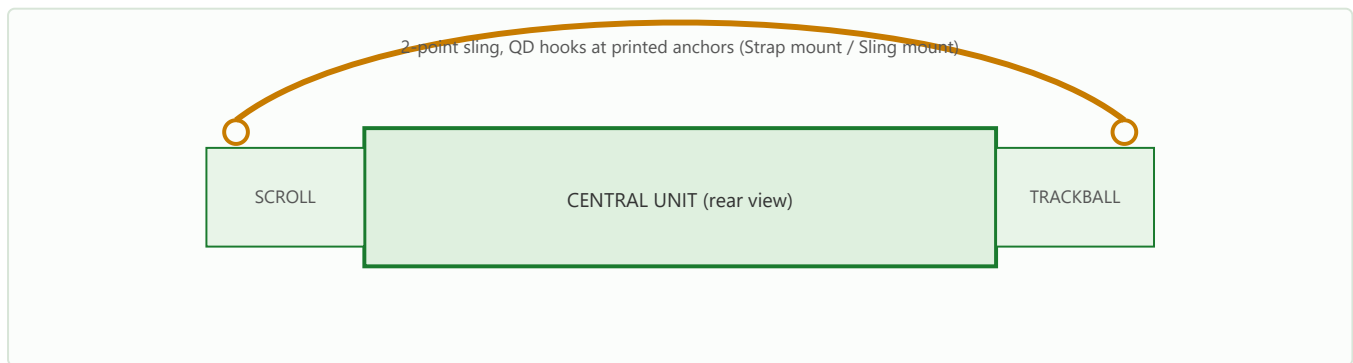


Fig. 6 — Sling load path: anchors on the two forward modules, load into the rails.

Check: deck hangs level on the sling; no clamp creep after 10 min of wear.

Step 9 — Comms module (optional): NFC + LoRa

Pi Pico • PN532 (NFC, I2C) • RFM9x LoRa (SPI, AS923) • SMA antenna • printed pod (body + lid)

The rail bus carries only USB, but the NFC and LoRa parts speak I2C and SPI — so they live in a removable pod with a **Raspberry Pi Pico** that reads them locally and presents to the deck as one USB serial device. Flash the Pico with the CircuitPython firmware in `os/upgrades/comms-module/firmware/`, wire the PN532 to I2C0 (GP4/GP5) and the RFM9x to SPI0 (SCK GP18, MOSI GP19, MISO GP16, CS GP17, RST GP20), and seat them in the printed `hardware-custom/Comms module/` body — PN532 behind the thinned NFC window, RFM9x at the SMA hole. Buy the **AS923 (920–925 MHz)** LoRa module: a US-915 / EU-868 unit is illegal to transmit on in Thailand and may not even tune. The firmware defaults to 923 MHz. The pod bolts to a printed picatinny clamp (the same clamp the other modules use — verify the 30 mm floor bolt spacing against your clamp before printing) and a short USB lead runs from the Pico to a rail USB port. Drive it from the deck with `deck-comms`.

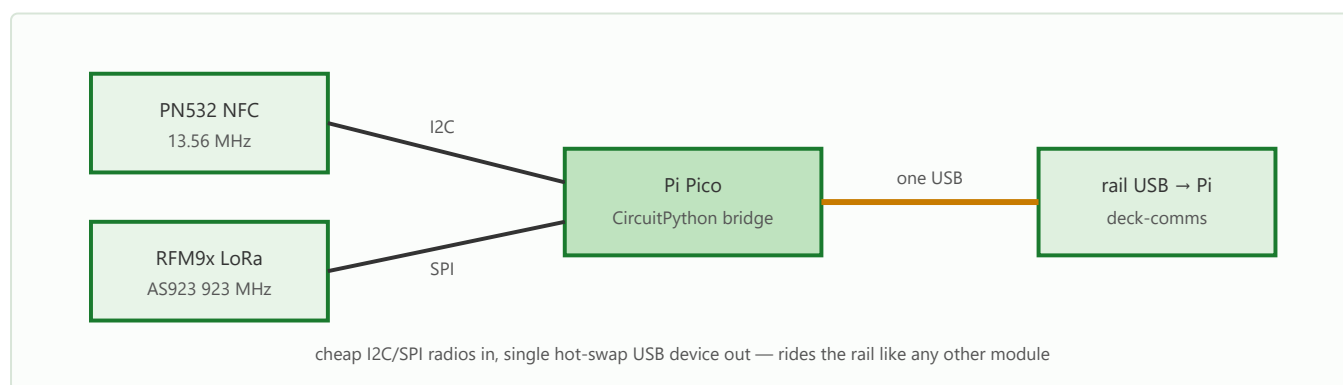


Fig. 7 — Comms module: a Pico bridges the I2C/SPI radios onto the USB rail bus.

Check: `deck-comms status` returns `NFC:ok LORA:ok`; `deck-comms nfc` prints a tag UID; two decks running `deck-comms lora tx / rx` hear each other.

Step 10 — Operating system: flash, inject, first boot

Windows PC • SD card reader • Raspberry Pi Imager • this repo

Flash *Raspberry Pi OS (64-bit) with desktop* using Raspberry Pi Imager; set hostname, user, Wi-Fi and SSH in Imager's customisation. With the card still mounted, run `os\image\inject-to-sd.ps1` — it copies the OS layer and chains into the first-boot mechanism. Boot 1 stages files offline and reboots; boot 2 runs the base install unattended — follow it with `journalctl -fu cyberdeck-firstboot`. The base layer is more than cosmetics: alongside FreeCAD, the zram memory system, the boot.d script runner and the green theme, it now hardens the deck (UFW firewall — default-deny inbound, SSH/Samba/Moonlight allowed from the LAN only; SSH key-only; kernel security sysctls) and tunes it for field work (CPU governor set per `deck-mode`, I/O scheduler chosen by drive type, `noatime` + delayed commit to spare the SD card).

On the booted deck, add the opt-in layers. The simplest path is the menu-driven installer `sudo ./os/install.sh` (a whiptail TUI — tick the layers you want); or run each idempotent script directly over SSH. What each layer adds:

Layer	Script	What it installs
Base (auto)	<code>setup.sh</code> / <code>install.sh</code>	FreeCAD, zram + VM tuning, boot.d, theme, UFW + SSH hardening , CPU-governor & storage I/O tuning
AI	<code>os/ai/setup-ai.sh</code>	Claude Code, memory-tuned Ollama, the <code>deckcoder</code> model, earlyoom
Extras	<code>os/extras/setup-extras.sh</code>	<code>deck-mode</code> , <code>deck-vault</code> (LUKS), conky overlay, RTC, SDR tools, <code>deck-ide</code> , <code>deck-check</code> , <code>deck-help</code>
Virtual	<code>os/extras/setup-virtual.sh</code>	Moonlight "borrowed GPU" + <code>deck-drive</code> (iSCSI / rclone cloud disks)
Upgrades	<code>os/upgrades/setup-upgrades.sh</code>	<code>deck-assistant</code> , USB HID, NAS, <code>deck-rfid</code> , <code>deck-lora</code> , <code>deck-comms</code> , and <code>deck-scroll</code> (scroll-handle → FreeCAD zoom / volume)
Apps	<code>apps/</code>	GRIMOIRE offline search + RAG, the <code>deck-*</code> dashboards/dev tools, your own programs

After installing, run `deck-check` to confirm the deck is healthy (hardware OpenGL not software fallback, zram active, governor and thermals sane) and `deck-help` to see every command. Put your own programs and downloaded apps in `apps/` — it is untouched by the installers and syncs to the deck via `deck-nas`, `deck-drive`, scp, or git; `apps/grimoire` gives you an offline, compressed full-text library (and RAG Q&A through the local model) for working without a network.

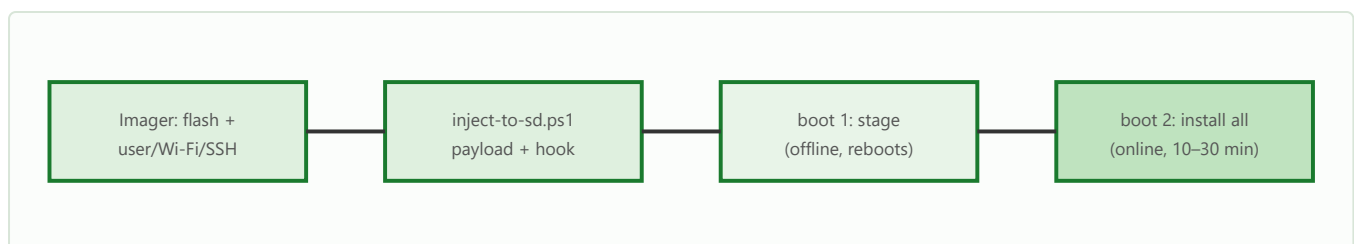


Fig. 8 — Zero-touch OS pipeline from Windows to a fully provisioned deck.

Check: login shows the DFCD banner; `swapon --show` lists `/dev/zram0`; `sudo ufw status` reads *default deny incoming*; `deck-check` reports a hardware OpenGL renderer (not `llvmpipe`) and no FAILs; FreeCAD launches.

Step 11 — Commissioning and acceptance

Subsystem	Test	Pass criterion
Health	<code>deck-check</code> on the desktop session	0 FAILs; summary green
GPU / render	<code>deck-check</code> GPU line (or <code>glxinfo -B</code>)	renderer is V3D /hardware GL, <i>not</i> llvmpipe software
Firewall	<code>sudo ufw status verbose</code>	default deny incoming; SSH/Samba/Moonlight allowed from LAN only
Thermals	<code>stress-ng --cpu 4</code> 10 min in the closed chassis	<code>temp</code> < 80 °C, no throttle flag in <code>vcgencmd get_throttled</code>
Power	Full FreeCAD session on battery	≥ 4 h runtime, clean soft-off from the power switch
Slide	100 open/close cycles	display + touch + USB stable throughout
Inputs	<code>evtest</code> per device	encoder, buttons, trackball, keyboard all enumerate and report
Scroll handle	<code>systemctl --user start deck-scroll</code> , turn the encoder	zooms in FreeCAD when focused; changes volume otherwise
Clock	Power off 1 h, boot without network	<code>timedatectl</code> shows correct wall time (RTC holds)
Status	<code>deck-mode stealth / work / bright</code>	LED + dimming follow each mode
Vault	<code>deck-vault init 2 && deck-vault open</code>	mount at <code>/mnt/vault</code> ; survives reboot locked
Radio	RTL-SDR + <code>rtl_fm -f 100.0M -M wbfm</code>	broadcast FM audible; AM via direct-sampling (V4)
Comms module	<code>deck-comms status</code> then <code>nfc / lora rx</code>	NFC:ok LORA:ok ; tag UID prints; LoRa packets received
USB HID	<code>sudo deck-hid on</code> into a host, <code>deck-hid type "hi"</code>	host receives the keystrokes (Pi 4/Zero; Pi 5 USB-C may not)
NAS	<code>deck-nas on</code> , open <code>\\<deck>\deck</code> from a PC	share mounts, files read/write over WiFi
Assistant	<code>deck-assistant "say hi"</code> with WiFi off	offline model answers locally
Offline library	<code>python apps/grimoire/backend/grimoire.py search "..."</code> with WiFi off	GRIMOIRE returns ranked hits from the compressed tome

When all rows pass, log the build date and any deviations in `CHANGELOG.md` — the repo is the deck's service manual from here on.